



Abstract Classes and Interfaces

Type	Lecture
Date	@January 14, 2022
Lecture #	1
Lecture URL	https://youtu.be/RiGkT9NDof4
Notion URL	https://21f1003586.notion.site/Abstract-Classes-and-Interfaces-e462336f5d97499d8c8b1d6d09ea045d
Week #	4

Abstract classes

- Provide an abstract definition of the method
- `public abstract double perimeter();`
- Forces sub-classes to provide a concrete implementation
- Cannot create objects from a class that has abstract functions
- Also, the class containing an abstract function must be declared abstract

```
public abstract class Shape {  
    ...  
    public abstract double perimeter();  
    ...  
}
```

- We can still declare variables whose type is an abstract class

```
Shape shapearr[] = new Shape[3];  
int sizearr[] = new int[3];  
  
shapearr[0] = new Circle(...);  
shapearr[1] = new Square(...);  
shapearr[2] = new Rectangle(...);  
  
for(int i = 0; i < 3; ++i) {  
    sizearr[i] = shapearr[i].perimeter()  
    // Here, each shapearr[i] calls the appropriate method  
    ...  
}
```

Generic Functions

- We can use abstract classes to specify generic properties

```
public abstract class Comparable {
    public abstract int cmp(Comparable s);
    // return -1 if this < s
    // return 0 if this == s
    // return 1 if this > s
}
```

- Now we can sort any array of objects that extend `Comparable`

```
public class SortFunctions {
    public static void quicksort(Comparable[] a) {
        ...
        // Code for quicksort goes here
        // Except that to compare a[i] and a[j], we use a[i].cmp(a[j])
    }
}
```

- To use this

```
public class Myclass extends Comparable {
    private double size; // Quantity used for comparison

    public int cmp(Comparable s) {
        if(s instanceof Myclass) {
            // Compare this.size and ((Myclass) s).size
            // We have to cast in order to access s.size
        }
    }
}
```

Multiple inheritance

- An interface is an abstract class with no concrete components

```
public interface Comparable {
    public abstract int cmp(Comparable s);
}
```

- A class that extends an interface is said to implement it

```
public class Circle extends Shape implements Comparable {
    public double perimeter() { ... }
    public int cmp(Comparable s) { ... }
    ...
}
```

We can extend only one class, but can implement multiple interfaces

Summary

- We can use the class hierarchy to group together related classes
- An abstract method in a parent class forces each subclass to implement it in a sensible manner
- Any class with an abstract method is itself abstract
 - Cannot create objects corresponding to an abstract class
 - However, we can define variables whose type is an abstract class
- Abstract classes can also describe capabilities, allowing for generic functions
- An interface is an abstract class with no concrete components
 - A class can extend only one parent class, but it can implement any number of interfaces



Interfaces

▼ Type	 Lecture
📅 Date	@January 14, 2022
☰ Lecture #	2
🔗 Lecture URL	https://youtu.be/adkU2TMbJfk
🔗 Notion URL	https://21f1003586.notion.site/Interfaces-7255fac9db6d4cb98f4961bb2b66f699
# Week #	4

Interfaces

- An interface is a purely abstract class
 - All methods are abstract
- A class **implements** an interface
 - Provide concrete code for each abstract function
- Classes can implement multiple interfaces
 - Abstract functions, so no contradictory inheritance
- Interfaces describe relevant aspects of a class
 - Abstract functions describe a specific “slice” of capabilities
 - Another class only needs to know about these capabilities

Exposing limited capabilities

- Generic `quicksort` for any datatype that supports comparisons
- Express this capability by making the argument type `Comparable[]`
 - Only information that quicksort needs about the underlying type
 - All other aspects are irrelevant
- Describe the relevant functions supported by Comparable objects through an interface
- However, we **cannot** express the intended behaviour of `cmp` explicitly

```
public class SortFunctions {
    public static void quicksort(Comparable[] a) {
        ...
    }
}
```



```

        // Code for quicksort goes here, except that to compare a[i] and a[j]
        // we use a[i].cmp(a[j])
    }
}

public interface Comparable {
    public abstract int cmp(Comparable s);
    // Return -1 if this < s;
    // Return 0 if this == s;
    // Return +1 if this > s;
}

```

Adding methods to interfaces

- Java interfaces extended to allow functions to be added
- Static functions
 - Cannot access instance variables
 - Invoke directly or using interface name: `Comparable.cmpdoc()`
- Default functions
 - Provide a default implementation for some functions
 - Class can override these
 - Invoke like normal method, using object name: `a[i].cmp(a[j])`

```

public interface Comparable {
    public static String cmpdoc() {
        String s;
        s = "Return -1 if this < s, ";
        s += "0 if this == s, ";
        s += "+1 if this > s.";
        return s;
    }
}

public interface Comparable {
    public default int cmp(Comparable s) {
        return 0;
    }
}

```

Dealing with conflicts

- Old problem of multiple inheritance returns
 - Conflict between static/default methods
- Subclass must provide a fresh implementation
- Conflict could be between a class and an interface
 - `Employee` inherits from class `Person` and implements `Designation`
 - Method inherited from the class "wins"
 - Motivated by reverse compatibility

```

public class Person {
    public String getName() {
        return "No name";
    }
}

public interface Designation {
    public default String getName() {
        return "No designation";
    }
}

public class Employee extends Person implements Designation {
    ...
}

```

Summary

- Interfaces express abstract capabilities
 - Capabilities are expressed in terms of methods that must be present
 - Cannot specify the intended behaviour of these functions
- Java later allowed concrete functions to be added to interfaces
 - Static functions — cannot access instance variables
 - Default functions — may be overridden
- Reintroduces the conflict in multiple inheritance
 - Subclass must resolve the conflict by providing a fresh implementation
 - Special "class wins" rule for conflict between superclass and interface
- Pitfalls of extending a language and maintaining compatibility



Private classes

▼ Type	 Lecture
📅 Date	@January 14, 2022
☰ Lecture #	3
🔗 Lecture URL	https://youtu.be/6eZ9mNX-GQ8
🔗 Notion URL	https://21f1003586.notion.site/Private-classes-d69d60fc19a24745a7d6c9bbf687d9d1
# Week #	4

Nested objects

- An instance variable can be a user defined type
 - `Employee` uses `Date`
- `Date` is a public class, also available to other classes
- When could a private class make sense?

```
public class Employee {
    private String name;
    private double salary;
    private Date joinDate;
    ...
}

public class Date {
    private int day, month, year;
    ...
}
```

- `LinkedList` is built using `Node`
- Why should `Node` be public?
 - May want to enhance with `prev` field, doubly linked list
 - Does not affect interface of `LinkedList`

```
public class Node {
    public Object data;
    public Node next;
    ...
}
```

```

public class LinkedList {
    private int size;
    private Node first;

    public Object head() {
        Object returnval = null;
        if(first != null) {
            returnval = first.data;
            first = first.next;
        }

        return returnval;
    }
}

```

- Instead, make `Node` a private class
 - Nested within `LinkedList`
 - Also called an inner class
- Objects of private class can see private components of enclosing class

```

public class LinkedList {
    private class Node {
        private Object data;
        private Node next;
        ...
    }

    private int size;
    private Node first;

    public Object head() { ... }

    public void insert(Object newData) { ... }
}

```

Summary

- An object can have nested objects as instance variables
- In some situations, the structure of these nested objects need not be exposed
- Private classes allow an additional degree of data encapsulation
- Combine private classes with interfaces to provide controlled access to the state of an object



Controlled interaction with Objects

Type	Lecture
Date	@January 14, 2022
Lecture #	4
Lecture URL	https://youtu.be/YefHD5_AGiw
Notion URL	https://21f1003586.notion.site/Controlled-interaction-with-Objects-0aa4802d22c749e3b156b477a8f9e078
Week #	4

Manipulating Objects

- Encapsulation is a key principle of object oriented programming
 - Internal data is private
 - Access to the data is regulated through public methods
 - Accessor and mutator methods
- Can ensure data integrity by regulating access

```
public class Date {
    private int day, month, year;

    public int getDay() { ... }
    public int getMonth() { ... }
    public int getYear() { ... }

    public void setDay(int d) { ... }
    public void setMonth(int m) { ... }
    public void setYear(int y) { ... }
}
```

- Update data as a whole, rather than individual components

```
public class Date {
    private int day, month, year;

    public int getDay() { ... }
    public int getMonth() { ... }
    public int getYear() { ... }
```



```

    public void setDate(int d, int m, int y) {
        ...
        // Validate the d-m-y combination
    }
}

```

Querying a database

- Object stores train reservation information
 - Can query availability for a given train, date
- To control spamming by bots, require the user to login before querying
- Need to connect the query to the logged in status of the user
- Interaction with state

```

public class RailwayBooking {
    private BookingDB railwayDB;

    public int getStatus(int trainno, Date d) {
        // Return the number of seats available
        // on train number trainno on date d
        ...
    }
}

```

- Need to connect the query to the logged in status of the user
- Use objects
 - On login, user receives an object that can make a query
 - Object is created from private class that can lookup `railwayDB`

```

public class RailwayBooking {
    private BookingDB railwayDB;

    private class QueryObject {
        public int getStatus(int trainno, Date d) {
            // Return the number of seats available
            // on train trainno on date d
            ...
        }
    }

    public QueryObject login(String u, String p) {
        QueryObject qobj;
        if(validLogin(u, p)) {
            qobj = new QueryObject();
            return qobj;
        }
    }
}

```

- How does the user know the capabilities of the private class `QueryObject` ?
- Use an interface
 - Interface describes the capabilities of the object returned on login

```

public interface QIF {
    public abstract int getStatus(int trainno, Date d);
}

public class RailwayBooking {
    private BookingDB railwayDB;
    public QIF login(String u, String p) {
        QueryObject qobj;
        if(validLogin(u, p)) {
            qobj = new QueryObject();
            return qobj;
        }
    }

    private class QueryObject implements QIF {
        public int getStatus(int trainno, Date d) {
            ...
        }
    }
}

```

```
    }  
  }  
}
```

- Query object allows unlimited number of queries
- Limit the number of queries per login
- Maintain a counter
 - Add instance variables to object returned on login
 - Query object can remember the state of the interaction

```
public class RailwayBooking {  
    private BookingDB railwayDB;  
    public QIF login(String u, String p) {  
        QueryObject qobj;  
        if(validLogin(u, p)) {  
            qobj = new QueryObject();  
            return qobj;  
        }  
    }  
  
    private class QueryObject implements QIF {  
        private int numQueries;  
        private final int QLIM;  
  
        public int getStatus(int trainno, Date d) {  
            if(numQueries < QLIM) {  
                // Respond, increment numQueries  
                ...  
            }  
            ...  
        }  
    }  
}
```

Summary

- Can provide controlled access to an object
- Combine private classes with interfaces
- External interaction is through an object of the private class
- Capabilities of this object are know through a public interface
- Object can maintain instance variables to track the state of the interaction

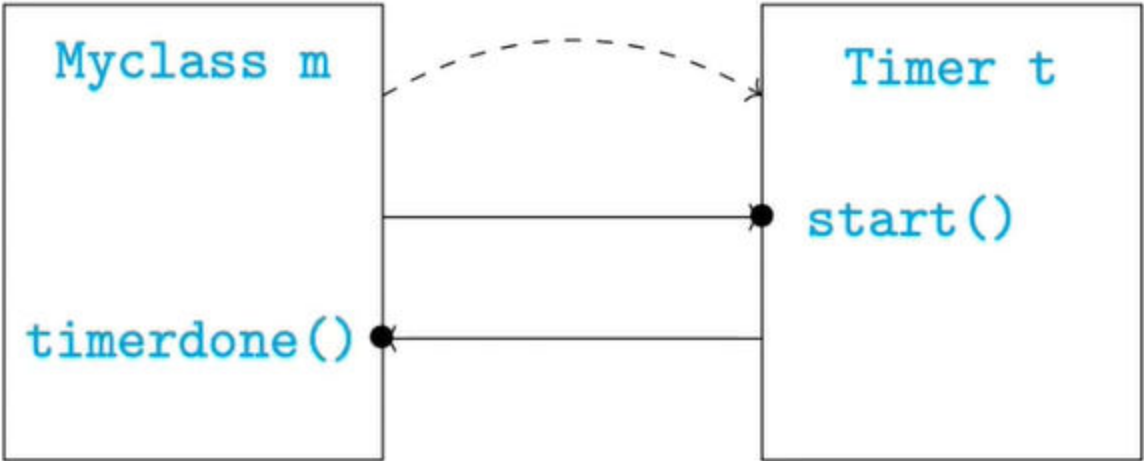


Callbacks

Type	Lecture
Date	@January 14, 2022
Lecture #	5
Lecture URL	https://youtu.be/CKjGnZCvIng
Notion URL	https://21f1003586.notion.site/Callbacks-3ed65be72c694d3d8d7b9d78bb505138
Week #	4

Implementing a call-back facility

- MyClass m creates a Timer t
- Start t to run in parallel
 - MyClass m continues to run
 - Will see later how to invoke parallel execution in Java!
- Timer t notifies MyClass m when the time limit expires
 - Assume MyClass m has a function timerDone()



Implementing Callbacks

- Code for MyClass
- Timer t should know whom to notify
 - MyClass m passes its identity when it creates Timer t

- Code for `Timer`
 - Interface `Runnable` indicates that `Timer` can run in parallel
- `Timer` is specific to `MyClass`
 - We need a generic `Timer`

```
public class MyClass {
    public void f() {
        ...
        Timer t = new Timer(this); // this object created t
        ...
        t.start(); // Start t
        ...
    }

    public void timerDone() { ... }
}

public class Timer implements Runnable {
    // Timer can be invoked in parallel
    private MyClass owner;
    public Timer(MyClass o) {
        owner = o; // Creator
    }

    public void start() {
        ...
        owner.timerDone();
    }
}
```

A generic timer

- A Java class hierarchy
- Parameter of `Timer` constructor of type `Object`
 - Compatible with all caller types
- Need to cast `owner` back to `MyClass`

```
public class MyClass {
    public void f() {
        ...
        Timer t = new Timer(this); // this object created t
        ...
        t.start(); // Start t
        ...
    }

    public void timerDone() { ... }
}

public class Timer implements Runnable {
    // Timer can be invoked in parallel
    private Object owner;
    public Timer(Object o) {
        owner = o; // Creator
    }

    public void start() {
        ...
        ((MyClass) owner).timerDone();
    }
}
```

Use interfaces

- Define an interface for callback
- Modify `MyClass` to implement `TimerOwner`
- Modify `Timer` so that owner is compatible with `TimerOwner`

```
public interface TimerOwner {
    public abstract void timerDone();
}

public class MyClass implements TimerOwner {
```



```

public void f() {
    ...
    Timer t = new Timer(this); // this object created t
    ...
    t.start(); // Start t
    ...
}

public void timerDone() { ... }
}

public class Timer implements Runnable {
    // Timer can be invoked in parallel
    private TimerOwner owner;
    public Timer(TimerOwner o) {
        owner = o; // Creator
    }

    public void start() {
        ...
        owner.timerDone();
    }
}

```

Summary

- Callbacks are useful when we spawn a class in parallel
- Spawned object notifies the owner when it is done
- Can also notify some other object when done
 - `owner` in `Timer` need not be the object that created the `Timer`
- Interfaces allow this callback to be generic
 - `owner` has to have the capability to be notified



Iterators

▼ Type	 Lecture
📅 Date	@January 14, 2022
☰ Lecture #	6
🔗 Lecture URL	https://youtu.be/BG_Btui0K1o
🔗 Notion URL	https://21f1003586.notion.site/Iterators-36904f8c2b464d8a87c7dcf96aed5d11
# Week #	4

Linear List

- A generic linear list of objects
- Internal implementation may vary
- An array implementation

```
public class LinearList {
    // Array implementation
    private int limit = 100;
    private Object[] data = new Object[limit];
    private int size;

    public LinearList() { size = 0; }

    public void append(Object o) {
        data[size++] = o;
        ...
    }
    ...
}
```

- A linked list implementation

```
public class LinearList {
    private Node head;
    private int size;

    public LinearList() { size = 0; }

    public void append(Object o) {
        Node m;
        for(m = head; m.next != null; m = m.next) {}

        Node n = new Node(o);
```

```

    m.next = n;
    size++;
}
...
private class Node { ... }
}

```

Iteration

- Want a loop to run through all the values in a linear list
- If the list is an array with public access, we could write this

```

int i;
for(i = 0; i < data.length; ++i) {
    ... // do something with data[i]
}

```

- For a linked list with public access, we could write this

```

Node m;
for(m = head; m != null; m = m.next) {
    ... // do something with m.data
}

```

- But we do not have public access
- And we do not know which implementation is in use either

Iterator

- Need the following abstraction

```

Start at the beginning of the list
while(there is a next element) {
    get the next element;
    do something with it
}

```

- Encapsulate this functionality in an interface called Iterator

```

public interface Iterator {
    public abstract boolean has_next();
    public abstract Object get_next();
}

```

- How do we implement `Iterator` in `LinkedList`?
- Need a pointer to remember position of the iterator
- How do we handle nested loops?

```

for(int i = 0; i < data.length; ++i) {
    for(int j = 0; j < data.length; ++j) {
        ... // do something with data[i] and data[j]
    }
}

```

- Solution → Create an `Iterator` object and export it

```

public class LinkedList {
    private class Iter implements Iterator {
        private Node position;
        public Iter() { ... }
        public boolean has_next() { ... }
        public Object get_next() { ... }
    }

    // Export a fresh iterator
    public Iterator get_iterator() {
        Iter it = new Iter();
    }
}

```

```
        return it;
    }
}
```

- Definition of `Iter` depends on the linear list

- Now, we can traverse the list externally as follows:

```
LinearList l = new LinearList();
...
Object o;
Iterator i = l.get_iterator();

while(i.has_next()) {
    o = i.get_next();
    ... // do something with o
}
...
```

- For nested loops, acquire multiple iterators

```
LinearList l = new LinearList();
...
Object oi, oj;
Iterator i, j;

i = l.get_iterator();
while(i.has_next()) {
    oi = i.get_next();
    j = l.get_iterator();

    while(j.has_next()) {
        oj = j.get_next();
        ... // do something with oi, oj
    }
}
...
```

Summary

- Iterators are another example of interaction with state
 - Each iterator needs to remember its position in the list
- Export an object with a pre-specified interface to handle the interaction
- The new Java `for` over lists implicitly constructs and uses an iterator

```
for(type x : a) {
    do something with x;
}
```